Library Management System

Interview Preparation Checklist

Spring Boot 3 | JPA / Hibernate | REST API | Target role: AEON Bank

How to use this sheet (30 min): Read Section 1 + 2 first (what you built & why) = 5 min. Memorise Section 3 (design patterns - they will ask names) = 5 min. Drill Section 4 (technical Q&A) = 10 min. Walk through Section 5 (live coding scenarios) = 10 min.

1. What We Built (recap of the assessment)

A REST API for a small library that manages three resources: Books (physical copies), Borrowers, and Borrowing (loans). Built with Spring Boot 3.2.5 / Java 17, Spring Data JPA, Bean Validation, and an H2 (dev/test) + PostgreSQL (prod) persistence layer.

Endpoints implemented

- POST /api/v1/books - Register a book copy (ISBN + title + author).
- GET /api/v1/books - List all books with per-copy availability flag.
- GET /api/v1/books/{id} - Get a single book copy by id.
- POST /api/v1/borrowers - Register a borrower (unique email).
- GET /api/v1/borrowers/{id} - Get a borrower by id.
- POST /api/v1/books/{id}/borrow - Borrow a book on behalf of a borrower.
- POST /api/v1/books/{id}/return - Return a book, freeing it for the next loan.

Key business rules satisfied

- One book copy can be borrowed by only one borrower at a time (409 on double-borrow).
- A registered borrower can borrow any available book copy.
- Multiple copies of the same ISBN are allowed, BUT title & author must match (409 on ISBN reuse for a different book).
- Returning a book frees it; the same copy can then be borrowed again.
- Borrower email is unique (case-insensitive, stored lowercased).

Non-functional work

- Layered architecture: Controller -> Service (interface + impl) -> Repository -> Entity.
- DTOs separate the API contract from the JPA entities (BookRequest/Response, etc.).
- Global exception handler (@RestControllerAdvice) -> one consistent error envelope.
- Bean Validation on all request DTOs (@NotBlank, @Email, @Pattern for ISBN).
- Pessimistic locking (SELECT ... FOR UPDATE) for concurrent borrow safety.
- OpenAPI / Swagger UI documentation via springdoc.
- Spring Boot Actuator health + info endpoints.
- Unit tests (Mockito) + integration tests (MockMvc @SpringBootTest) + a concurrency test.
- JaCoCo code coverage, Maven Surefire + Failsafe split (unit vs *IT integration tests).
- Multi-stage Dockerfile (build on maven image, run on JRE, non-root user).
- docker-compose.yml wiring app + Postgres; GitHub Actions CI pipeline.

2. Why We Chose These Technologies / Approaches

- Spring Boot 3 + Java 17: industry standard, fast startup, opinionated defaults; Java 17 gives records, sealed classes, pattern matching, modern switch.
- Spring Data JPA: removes boilerplate CRUD; repository interfaces auto-generate SQL. Lets us hand-write @Query + @Lock where we need control (pessimistic locking).
- H2 in-memory for dev/test, PostgreSQL for prod: zero-friction local dev + a real RDBMS in prod. Same code, profile-driven datasource via application-{profile}.yml.
- Pessimistic (not optimistic) locking for borrow: two librarians clicking 'borrow' on the same copy at once MUST not both succeed. SELECT FOR UPDATE on the book row serialises them; optimistic @Version would retry/conflict but the borrow is a one-shot user action.

- DTO + Response.from() static factory: keeps entities internal; we never expose JPA proxies to the API, avoiding lazy-initialisation exceptions and accidental field leaks.
- @RestControllerAdvice + ApiResponse envelope: every error has the same shape (timestamp, status, error, message, fieldErrors) - easy for clients to parse.
- BaseEntity @MappedSuperclass: id, createdDate, lastModifiedDate, @Version are shared by all entities - no duplication, consistent auditing.
- springdoc OpenAPI: live Swagger UI at runtime; annotations double as documentation.
- Failsafe for *IT tests: keeps slow integration tests out of the unit-test phase, matching Maven convention; CI runs 'mvn verify' so both execute.
- Multi-stage Docker + non-root user: smaller image, no JDK in runtime, security best practice.

3. Technical Designs / Design Patterns Used (memorise the names)

There are roughly 10 named patterns/principles in this codebase. Interviewers love names:

- 1. Layered (n-tier) Architecture - Controller / Service / Repository / Entity separation.
- 2. DTO Pattern (Data Transfer Object) - BookRequest, BookResponse, etc. isolate the API.
- 3. Service Interface + Implementation - BookService (interface) + BookServiceImpl. Enables mocking, swap of impl, program-to-interface.
- 4. Repository Pattern - Spring Data JPA repositories abstract persistence.
- 5. Builder Pattern - Lombok @Builder on DTOs / ApiResponse; fluent object construction.
- 6. Static Factory Method - BookResponse.from(book, available), BorrowRecordResponse.from(record).
- 7. Front Controller - Spring DispatcherServlet routes all requests (framework-provided).
- 8. Centralised / Global Exception Handling - @RestControllerAdvice + @ExceptionHandler.
- 9. Template Method / Inheritance - BaseEntity @MappedSuperclass holds common audit fields.
- 10. Pessimistic Locking (concurrency control) - @Lock(PESSIMISTIC_WRITE) / SELECT FOR UPDATE.
- Bonus: Optimistic Locking primitive present - @Version in BaseEntity (JPA optimistic lock).
- Bonus: Dependency Injection / Constructor injection (no @Autowired on fields - testable).
- Bonus: Single Responsibility - ApiConstants, FieldError each do one thing.

4. Likely Technical Questions (AEON Bank - financial context)

Architecture & general

- Q: Walk me through your architecture. A: Layered Spring Boot app. Controllers expose REST under /api/v1, delegate to service interfaces, which call JPA repositories. DTOs decouple the API from entities. A global advice normalises errors. Postgres in prod, H2 in test.
- Q: Why interfaces for services when there's one impl? A: Program to interface; easier mocking in unit tests, clear contract, room to add a second impl (e.g. caching) later.
- Q: How would you scale this for AEON Bank's traffic? A: Stateless app -> horizontal scale behind a load balancer; externalise session (none here, good); add a cache (Redis) for hot reads like book listings; read replicas for GETs; DB connection pooling (Hikari already in).

Concurrency (most likely deep-dive)

- Q: Two users borrow the same book at once - what happens? A: Pessimistic write lock on the book row (SELECT ... FOR UPDATE). Second request blocks until first commits, then sees the active loan and gets 409. Exactly one succeeds - proven by BorrowingConcurrencyIT.
- Q: Why lock the book row, not the loan row? A: When the copy is free, the active-loan query returns no rows and locks nothing (Postgres has no gap locking) - both requests would slip through. Locking the book row always acquires a lock, serialising the check-then-insert.
- Q: Pessimistic vs optimistic locking? A: Pessimistic locks upfront (good for high contention, like borrowing). Optimistic uses @Version, no lock, retries on conflict - good for low contention. We have @Version in BaseEntity as the optimistic primitive too.

- Q: What if the lock holder crashes? A: The DB transaction rolls back, lock is released, blocked request proceeds. Set a lock timeout to avoid indefinite waits.

JPA / Hibernate

- Q: N+1 problem - did you handle it? A: Yes. The book list fetches borrowed book ids in ONE query (findBookIdsWithActiveLoan) and maps availability from the Set, instead of one query per book.
- Q: Why LAZY on BorrowRecord.book/borrower? A: Avoid loading related entities unless needed; prevents eager joins on every loan read. DTO uses .getId() so no lazy init issue.
- Q: ddl-auto=update in prod - ok? A: No for real prod. I'd use Flyway/Liquibase with ddl-auto=validate. Left as update here to keep the exercise self-contained (noted in code).

REST & validation

- Q: How are validation errors returned? A: @Valid triggers MethodArgumentNotValidException; the advice maps it to 400 with a fieldErrors array (field, message, objectName, rejectedValue).
- Q: HTTP status codes used? A: 201 create, 200 read/return, 400 validation, 404 not found, 409 conflict (double-borrow / ISBN reuse), 405 method not supported, 500 unhandled.
- Q: Is your API idempotent? A: Borrow is not idempotent (calling twice = two loans, but second is blocked by the active-loan check). Return is effectively idempotent - returning an already-returned book yields 404 'no active loan'. Could add an idempotency-key header.

Testing

- Q: Test strategy? A: Unit tests (Mockito, no Spring) for service logic; integration tests (MockMvc + real H2) for controller->db; a CountDownLatch concurrency test proving the lock.
- Q: Why failsafe? A: Surefire runs unit tests; *IT classes run in integration-test phase via Failsafe. 'mvn verify' runs both; 'mvn test' runs only unit tests.

5. Live-Coding Scenarios ('improve a feature - show me the code')

A) Add Security to the API (most asked for a bank)

Approach: spring-boot-starter-security + JWT. Stateless, role-based (LIBRARIAN vs BORROWER).

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
  @Bean
  public SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.csrf(c->c.disable())
      .sessionManagement(s->s.sessionCreationPolicy(STATELESS))
      .authorizeHttpRequests(a -> a
        .requestMatchers("/api/v1/books/**","/api/v1/borrowers/**").hasRole("LIBRARIAN")
        .requestMatchers("/api/v1/books/*/borrow","/api/v1/books/*/return").hasRole("BORROWER")
        .anyRequest().authenticated())
      .oauth2ResourceServer(o->o.jwt(Customizer.withDefaults()));
    return http.build();
  }
}
```

- Mention: input validation (already done), rate limiting (Bucket4j/Resilience4j), HTTPS/TLS, secrets in a vault (not in docker-compose), audit logging of borrows.

- For AEON: PCI-DSS / data encryption at rest, PII handling for borrower email, OWASP Top 10 (injection - we use parameterised queries via JPA, so safe).

B) Add Pagination + Sorting to GET /books

```
@GetMapping
public ResponseEntity<Page<BookResponse>> getAll(
    @RequestParam(defaultValue="0") int page,
    @RequestParam(defaultValue="20") int size,
    @RequestParam(defaultValue="title") String sort,
    @RequestParam(defaultValue="asc") String dir) {
  Pageable p = PageRequest.of(page, size,
      Sort.by(Sort.Direction.fromString(dir), sort));
  return ResponseEntity.ok(bookService.getAll(p));
}
```

```
// repository + service
Page<Book> findAll(Pageable p);   // already in JpaRepository
```

C) Add a 'search by title/author' endpoint

```
// BookRepository
Page<Book> findByTitleContainingIgnoreCaseOrAuthorContainingIgnoreCase(
    String title, String author, Pageable p);
```

- Or a JPA Specification for dynamic multi-field search; or full-text if scaling up.

D) Add overdue-loan tracking + scheduled job

```
@Entity
public class BorrowRecord {
  private LocalDate dueDate;        // borrowedAt + 14 days
  private boolean overdue;
}
@Scheduled(cron = "0 0 1 * * *")
public void markOverdueLoans() {
  repo.findActiveLoansDueBefore(LocalDate.now())
      .forEach(r -> r.setOverdue(true));
}
```

- Enable with @EnableScheduling; add a fine/notification event for AEON-style reminders.

E) Add caching for hot reads (Redis)

```
@Cacheable(value="books", key="#id")
public Book getById(Long id) { ... }
@CacheEvict(value="books", key="#book.id")
public Book register(Book book) { ... }
```

- Mention cache invalidation on borrow/return; TTL; Caffeine for local, Redis for distributed.

F) Add API versioning / idempotency

- Versioning: path (/api/v1) already done; could add header or content negotiation.
- Idempotency: accept Idempotency-Key header, store request+response in Redis for 24h; replay same response on duplicate. Critical for payment-style endpoints at a bank.

G) Add a DB migration tool (Flyway)

- Add flyway-core dep, set spring.flyway.enabled=true, ddl-auto=validate. Put V1__init.sql in src/main/resources/db/migration. Every schema change = a new versioned, reviewed SQL file.

6. 30-Second 'Elevator' Answers (if they ask quickly)

- Biggest challenge: 'Concurrent borrowing. I chose pessimistic locking on the book row because optimistic locking would let a retry through, and locking the loan row fails when the copy is free. A CountDownLatch test proves exactly one of two simultaneous borrows wins.'
- Trade-off you accepted: 'ddl-auto=update in prod for exercise simplicity; I flagged it and would replace it with Flyway +

validate in a real release.'

- What you'd improve next: 'Add Spring Security + JWT, pagination, Redis caching for book listings, Flyway migrations, and an idempotency key on borrow/return.'
- Why this fits AEON Bank: 'Same concerns a bank API has - concurrency on shared state (like an account balance), audit trails, idempotent money-movement, strict validation, and security. The borrow flow is structurally similar to a debit transaction.'

Tip: if asked to code live, narrate the trade-off out loud (why pessimistic, why DTO, why @Transactional). Interviewers score reasoning as much as the code. Good luck!